Software Programming

IGS1931

Week 11: Files

Save It, Load It, Find It



HK©2026

Today

Activity	In class
Review	Where files live and how Python opens, writes, and reads them
Practice	Predict outputs and fix three bugs
Save and load	Write a list to a file and read it back
Detective case	Solve the case with your project team
Debrief	Review three common file bugs
Quiz	i-Class · Weeks 9–11 (strings, lists, files)

Open the Week 11 Colab badge in the README and run the setup cell.

Why Files?

```
scores = [70, 95, 60]     # stored in RAM while the program runs
```

	Variables (RAM)	Saved files (disk / SSD)
Speed	Very fast	Slower
After the program ends	Gone	Still there
After a computer restart	Gone	Still there
Example	scores = [70, 95, 60]	scores.txt

Final project requirement

Save data before the program closes and load it again the next time it runs.

Where Do Files Live? Folder Trees

A path names the folders you pass through to reach a file.

Windows

```
C:\
└── Users\
    └── mina\
        ├── Desktop\
        └── Documents\
            └── project\
                ├── main.py
                └── scores.txt
D:\  (another drive)
```

`C:\Users\mina\Documents\project\scores.txt`

macOS

```
/
└── Users/
    └── mina/  (~)
        ├── Desktop/
        └── Documents/
            └── project/
                ├── main.py
                └── scores.txt
```

`/Users/mina/Documents/project/scores.txt`

Linux / Colab

```
/
├── home/
│   └── mina/  (~)
│       └── project/
│           └── scores.txt
└── content/  (Colab)
    ├── harrypotter.txt
    └── sample_data/
```

`/home/mina/project/scores.txt`

	Windows	macOS	Linux / Colab
Root and separator	C:\, D:\ (backslash)	/ (slash)	/ (slash)
Scores.txt vs scores.txt	Usually the same file	Usually the same file	Usually different files
Extensions in file browser	Often hidden by default	Visibility varies	Visibility varies

Absolute and Relative Paths

Absolute path · starts at the root

```
C:\Users\mina\Documents\project\scores.txt     /Users/mina/Documents/project/scores.txt
```

Relative path · starts at the current working directory (cwd)

```
import os
print(os.getcwd())         # /content in Colab
open('scores.txt')         # in the cwd
open('data/scores.txt')    # down into data/
open('../scores.txt')      # up one folder
```

```
Documents/
└── project/  ← cwd
    ├── scores.txt
    └── data/
        └── scores.txt
```

Windows paths in Python strings

```
'C:\new\test.txt'     # \n becomes a newline and \t becomes a tab
'C:\\new\\test.txt'   # double backslashes
r'C:\new\test.txt'    # raw string
'C:/new/test.txt'     # forward slashes also work on Windows
os.path.join('data', 'scores.txt')
```

Anatomy of open()

```
f = open('output.txt', 'r')
```

f · file object | **'output.txt' · filename or path** | **'r' · mode**

open('output.txt') uses 'r' (read) by default.

1. Open → **2. Read or write** → **3. Close**

Setup creates harrypotter.txt and the show() helper.

The Four Modes (1.1)

Mode	Meaning	File is missing	File exists
`'r' (default)`	Read	FileNotFoundError	Read from the start
`'w'`	Write	Create it	Erase contents, then write
`'a'`	Append	Create it	Add to the end
`'x'`	Create	Create it	FileExistsError

What will be in output.txt after each step?

```
f = open('output.txt', 'w'); f.write('This is the first line in the file\n'); f.close()
f = open('output.txt', 'a'); f.write('This is the 2nd line in the file\n');   f.close()
f = open('output.txt', 'w'); f.write('Is this the 3rd line in the file?\n');  f.close()
```

After 'w'

```
This is the first line in the file
```

After 'a'

```
This is the first line in the file
This is the 2nd line in the file
```

After 'w' again

```
Is this the 3rd line in the file?
```

Writing: Three Rules (1.2)

1 **Add your own newline**

write() never adds '\n'.

2 **Pass a string**

Use str(95) before writing a number.

3 **Check the return value**

write() returns the character count.

```
f = open('demo.txt', 'w')
n = f.write('score: ' + str(95) + '\n')  # n is 10
f.write('no newline here')
f.write('...so this sticks to it\n')
f.close()
```

What does demo.txt contain?

```
score: 95
no newline here...so this sticks to it
```

A newline counts as one character in the value returned by write().

Reading: The Cursor (1.3)

Every read starts where the previous read stopped.

Start

```
|Harry Potter is…
```

After read(5)

```
Harry| Potter is…
```

After read(7)

```
Harry Potter| is…
```

After readline()

```
|The novels…
```

Code	Returns	Cursor moves
`f.read()`	The rest of the file as one string	To the end
`f.read(5)`	The next 5 characters	5 characters forward
`f.readline()`	Rest of the current line, including '\n' if present	To the next line or EOF
`for line in f:`	Each remaining line, including '\n' if present	Line by line to the end

At the end of the file, f.read() returns '' (the empty string).

Strings In, Strings Out (1.3)

Writing a number

```
95  →  str(95) + '\n'  →  file
```

Reading it back

```
file  →  '95\n'  →  '95'  →  95
```

```
f = open('harrypotter.txt')
first = f.readline()
f.close()
print(repr(first))
print(first.strip())
```

Task	Code
Write a number	`str(x) + '\n'`
Strip surrounding whitespace	`line.strip()`
Convert a line to a number	`int(line.strip())`
Print without an extra newline	`print(line, end='')`

What will be printed?

```
'Harry Potter is a series of fantasy novels.\n'
Harry Potter is a series of fantasy novels.
```

repr() makes '\n' visible. Text-mode reads return strings.

Closing Files and with (1.4)

Closing flushes buffered writes and releases the file handle.

Close explicitly

```
f = open('harrypotter.txt')
for line in f:
    print(line, end='')
f.close()
```

Close automatically

```
with open('harrypotter.txt') as f:
    for line in f:
        print(line, end='')
# closed automatically here
```

with closes the file when the block ends, even if an error occurs.

```
print(f.closed)      # True
```

On Windows, an open file can also prevent renaming or deleting it.

File Errors: try / except (1.4)

Error	When it happens
`FileNotFoundError`	Read mode ('r') opens a file that is missing
`FileExistsError`	Create mode ('x') opens a file that already exists

Handle the error

```
try:
    with open('harrypotter.txt') as f:
        for line in f:
            print(line, end='')
except:
    print('File reading error!')

try:
    fin = open('bad_file')
except:
    print('Something went wrong.')
```

Check for a save file first

```
import os

if os.path.exists('scores.txt'):
    with open('scores.txt') as f:
        print(f.read())
else:
    print('No saved scores yet. Starting fresh!')
```

For a missing save file, use except FileNotFoundError: to handle that specific error.

The os Toolbox (1.5)

```
import os
```

/content → notes/ → todo.txt

Code	What it does	Example result
`os.getcwd()`	Get the working directory	`'/content'`
`os.listdir('notes')`	List names inside a folder	`['todo.txt']`
`os.path.join('notes', 'todo.txt')`	Join folder and name	`'notes/todo.txt'`
`os.path.exists(p)`	Check whether the path exists	`True / False`
`os.path.isfile(p) / os.path.isdir(p)`	Check file type or folder type	`True / False`
`os.path.abspath(p)`	Get the absolute path	`'/content/notes/todo.txt'`
`os.mkdir('notes')`	Create a folder	
`os.remove(p) / os.rmdir('notes')`	Delete a file / delete an empty folder	

listdir returns names. Join each name to its folder before calling isfile.

File Cheat Sheet

Task	Syntax and reminders
Open	`with open(path, mode) as f:`
Modes	`'r' read · 'w' overwrite · 'a' append · 'x' create new`
Write	`f.write(str(x) + '\n')`
Read	`f.read(), f.read(n), f.readline(), for line in f:`
Convert	`line.strip(), int(line.strip())`
Handle errors	`FileNotFoundError ('r'), FileExistsError ('x'), try / except`
Build paths	`os.getcwd(), os.path.join(folder, name), r'C:\new\test.txt'`
Inspect folders	`listdir, isfile, isdir, exists, abspath`
Change folders	`mkdir, remove, rmdir (empty folders only)`

Reads continue from the current cursor. write() needs a string and an explicit newline.

What Will Be Printed? (Part 2)

Predict 1 · Reading in pieces

```
with open('harrypotter.txt') as f:
    print(f.read(5))
    print(f.read(7))
    print(f.readline(), end='')
    for line in f:
        print(line)
```

```
Harry
 Potter
 is a series of fantasy novels.
The novels chronicle the lives of a young wizard.

The main story arc concerns Harry and Voldemort.
```

Predict 2 · When opening goes wrong

```
print(os.path.exists('output.txt'),
      os.path.exists('thisfile.txt'))
try:
    open('output.txt', 'x')
except FileExistsError:
    print('x: output.txt already exists')
try:
    open('thisfile.txt', 'r')
except FileNotFoundError:
    print('r: thisfile.txt does not exist')
```

```
True False
x: output.txt already exists
r: thisfile.txt does not exist
```

Bug Hunt: Three Classic File Bugs (Part 3)

Run each buggy cell and read the output before changing the code.

	Bug	Symptom	Fix
1	`f.write(name)`	`'AzizMinaJun'`	`f.write(name + '\n')`
	See slide 8		
2	`with open(filename, 'w')` inside the loop	`Only 'logout\n' remains`	`Open once before the loop.`
	See slide 7		
3	`total = total + line`	`TypeError: int + str`	`total += int(line.strip())`
	See slide 10		

Save and Load (Part 4)

	Exercise	Expected result
E1	`save_lines('scores.txt', [70, 95, 60])`	`'70\n95\n60\n'`
E2	`load_lines('fruits.txt')`	`['banana', 'apple', 'mango']`
	`load_lines('missing.txt')`	`[]`

Run the last notebook cell three times

```
tasks = load_lines('tasks.txt')
tasks.append('task ' + str(len(tasks) + 1))
save_lines('tasks.txt', tasks)
```

Run 1

```
['task 1']
```

Run 2

```
['task 1', 'task 2']
```

Run 3

```
['task 1', 'task 2', 'task 3']
```

Notebook · README → Week 11 Colab badge

The Missing Proposal (Part 5)

The night before the deadline, someone deleted Team 7’s proposal from the LAB-3 PC. The security system left the evidence in a folder named case.

Room 1 · Files
`listdir + isfile`

→

Room 2 · Badge
`camera.log`

→

Final · Culprit
`badges/<badge>.txt`

1. Sit with your project team. Use one laptop.
2. Run the engine cell, then start('team name').
3. Call unlock('files', ...). If it fails, read the clue and try again.
4. Each hint('files') adds 1 minute to your time.
5. Call solve(name) and show the result to the TA. Fastest time wins.

Debrief: Three File Traps (Part 6)

Trap	What happens	Fix
`open(name, 'w')`	Clears the file each time it opens	Open once, or use 'a' to append
`f.write(x)`	Needs a string and adds no newline	f.write(str(x) + '\n')
`Reading numbers`	Each line comes back as a string	int(line.strip())

Fastest detectives

Next time · Graphical interfaces with tkinter

Use a local Python environment such as VS Code or PyCharm.

Quiz

Open i-Class

10 questions · Weeks 9–11 · Strings, lists, and files

Output What does the code print?

Debug Which change fixes the bug?

Close the notebook. Read the code without running it.

Remember the cursor, what 'w' does, and the type returned by a text read.